

RealEstate Hub AI Voice Agent

Final Capstone Report

Project: Week 4 Capstone — Production-Grade AI Voice Agent
Domain: Real Estate / Conversational AI
Date: September 25, 2026

48/48 Tests Passed	12/12 Injectons Blocked	412 ms Latency p95	90% Retrieval Accuracy
--	---	--	--

Executive Summary

A production-ready AI voice agent (“Ayesha”) for a Pakistani real estate company that speaks natural UrduLish, answers property questions using retrieval-augmented generation, recommends properties, handles objections, and books appointments end-to-end. Built with a modern AI stack combining Groq LLM, Deepgram STT, Fish Audio TTS, LangGraph orchestration, ChromaDB vector search, and FastAPI backend.

Key Results

- ✓ 48/48 test conversations passed (100%)
- ✓ 12/12 prompt injection attacks blocked (100%)
- ✓ Latency p95: 412 ms (target < 2000 ms)
- ✓ Retrieval accuracy: 90%
- ✓ 4/4 API tests passed

Day 1 — Foundations of AI Voice Agents & Conversation Design

Architecture

A production voice agent pipeline with 8 layers: Telephony → STT → LLM Reasoning → Tool Calling → RAG Retrieval → Memory → TTS → Orchestration.

The Workflow Orchestration layer (LangGraph) ties these together as a state machine rather than a one-shot chatbot.

Call Sequence per Turn

Caller speaks → STT produces transcript → LangGraph updates state → LLM decides (answer / RAG / tool call / clarify) → TTS synthesizes → Telephony streams back to caller → Memory persists turn.

Latency budget: <2 seconds end-to-end (STT + LLM + tool + TTS first chunk).

Conversation Flows (7 Designed)

- Buyer inquiry — budget, city, area → RAG+SQL → 2-3 options → objection → visit booking
- Rental inquiry — monthly budget, city, duration → rental search → booking
- Commercial — business type, area, budget → commercial listings + zoning
- Investment — budget, ROI goals → developer track record + appreciation
- Returning customer — CRM phone lookup → greet by name → recall preferences
- Reschedule — lookup appointment → new slot → update calendar + email
- Cancel — lookup → confirm → cancel calendar + email → offer reschedule

UrduLish Persona Engineering

Persona: “Ayesha” — warm, professional, Pakistani property consultant.

Traits: Pakistani-natural, professional, warm, persuasive, patient.

Scripted Phrases

- Greeting: “Assalam-o-Alaikum sir! RealEstate Hub se baat ho rahi hai, main Ayesha.”
- Acknowledgement: “Ji bilkul”, “Acha samajh gayi”, “Theek hai noted”
- Hesitation: “Ek second sir...”, “Hmm... acha”, “Thori dair dijiye...”
- Objection handling: “Main samajh sakti hoon... [empathy] ... [evidence]”

Guardrail: Never translate English word-for-word. Keep real estate nouns in English (plot, possession, booking, installment).

Fish Audio Evaluation

Chosen over ElevenLabs for:

- Lower latency streaming (fits <2s turn budget)
- Better Urdu-English code-switching
- Natural Urdu pronunciation
- Competitive pricing
- Strong voice cloning

System Prompt

Production-grade system prompt with:

- Scope: Property buying/renting/commercial/investment, recommendations, objection handling, scheduling
- Guardrails: Ground every fact in retrieved data; never fabricate; ignore prompt injections; never share internal data
- Persuasion rules: Empathy first, evidence second, no pressure
- Appointment policy: Only book via Calendar tool, confirm details, notify employee via Email tool
- Escalation: Legal/tax/complaints → human agent

Day 1 Deliverables

- ✓ Architecture diagram (8-layer pipeline)
- ✓ 7 conversation flowcharts
- ✓ UrduLish persona + phrase bank
- ✓ Fish Audio vs ElevenLabs evaluation
- ✓ Production system prompt

Day 2 — Knowledge Base, RAG & Property Intelligence

Dataset Summary

- Total properties: 700 (sampled from Kaggle Zameen.com dataset)
- Cities: Lahore, Karachi, Islamabad, Rawalpindi, Faisalabad
- Split: ~500 For Sale, ~233 For Rent
- Supplementary tables (synthetic): amenities, schools, hospitals, payment_plans, faqs

Retrieval Accuracy

- Score: 90.0% (18/20)
- Method: 20 test questions built from real data; measured whether expected facts appeared in retrieved context
- Single miss: “5 bedroom house in Bahria Town Lahore around 20,500,000 rupees” — exact combination not in sample

Chunk-Size Evaluation

Chunk Size	Overlap	Precision@1	Avg Chunks/Property	Total Chunks
150	20	66.67%	5.04	3526
400 (chosen)	60	80.00%	2.00	1401
800	100	73.33%	1.00	700

Chosen: 400 chars / 60 overlap.

Groundedness

- Raw score: 0/5 (Unicode narrow no-break space caused regex mismatch — false alarm)
- After whitespace normalization fix: 5/5 grounded

SQL vs Vector Split

- SQL (properties.db): price, availability, plot size, bedrooms, baths, agency, agent
- Vector (ChromaDB): amenities, schools, hospitals, payment plans, FAQs, descriptions

Sample Working Queries

Q: “3 bedroom house in DHA Lahore”

A: Ji sir, DHA Defence mein 3-bedroom ke teen options hain:

- property_id 13753525 – 5 Marla, PKR 15 million, 3 bed/5 bath
- property_id 15740422 – 5 Marla, PKR 22.5 million, 3 bed/4 bath
- property_id 15178969 – 1 Kanal, PKR 24 million, 3 bed/4 bath

Q: “What is the minimum down payment for an installment plan?”

A: Ji sir, installment plans mein down payment 10% se 25% tak ho sakti hai...

Key Day 2 Numbers

- Retrieval Accuracy: 90%

- Optimal chunk size: 400 chars, 60 overlap
- Precision@1: 80%
- Properties in SQL: 700
- Vector chunks: 1401
- FAQs: 15

Day 3 — Voice Agent & Natural Conversation

Task 1: Streaming Voice Pipeline

- STT (simulated 220ms) + LLM (measured) + TTS (simulated 300ms)
- Latency budget target: <2000ms per turn
- Cold start: ~3-5s (one-time, models loaded at service startup in production)

Task 2: Natural Speech Behaviors

Implemented fillers, hesitation, thinking pauses, acknowledgements:

- Thinking pauses: “Ek second sir...”, “Hmm... acha”, “Thori dair dijiye...”
- Acknowledgements: “Acha samajh gayi”, “Ji bilkul”, “Theek hai noted”
- Context-aware: Only insert filler when action requires lookup

Task 3: Context Memory

Slots tracked across turns:

- Budget (parses “3 crore”, “50 lakh”)
- City (Lahore, Karachi, Islamabad, Rawalpindi, Faisalabad)
- Location hint (DHA, Bahria, Gulberg, etc.)
- Bedrooms
- Purpose (For Sale / For Rent)
- Intent (buyer / renter / investor / commercial)

Special handling: “Us se sasti” → auto-lowers budget by 15%.

Task 4: Objection Handling (6 Types)

Category	Keywords	Response Strategy
Price	expensive, mehnga, zyada	Empathy + alternative + installment plan
Trust	fraud, dhoka, scam	Verified partner + track record
Location	door, far, developing	Infrastructure + appreciation
Investment	roi, resale, future	Historical trends + comparables
Builder	developer, handover	Past project list
Maintenance	upkeep, society fee	Reasonable fee explanation

Task 5: Human Evaluation

Criterion	Average
Naturalness	4.0 / 5
Persuasiveness	4.0 / 5
Fluency	4.0 / 5
Conversation Flow	3.8 / 5

Strongest flow: Buyer (DHA Lahore) — 5/5 across all criteria

Weakest flow: Cancel — 3/5 (middle turn asked buying question)

Day 3 Deliverables

- ✓ Voice pipeline with latency breakdown
- ✓ Natural speech behaviors
- ✓ Context memory across turns
- ✓ 6 objection types detected + answered
- ✓ Human evaluation transcript + CSV scores

Day 4 — Workflows, Scheduling & Business Automation

Deliverables

- day4/src/appointments.py — SQLite appointment store with availability check
- day4/src/calendar_integration.py — Google Calendar API (mock/live modes)
- day4/src/email_integration.py — SMTP email notifications
- day4/src/crm.py — Call logs, preferences, follow-up reminders
- day4/src/workflow.py — Full flow: intent → match → book → calendar → email → CRM
- day4/src/n8n_workflow.json — Visual n8n equivalent

Tests Passed

- ✓ Double-booking prevention (SQLite overlap check)
- ✓ Reschedule + cancel synced across DB, Calendar, Email
- ✓ Full workflow: 6/6 steps ok, outcome=booked
- ✓ Retry with linear backoff

Artifacts

- day4/data/crm.db — appointments + call_logs + follow_up_reminders
- day4/data/calendar_log.jsonl — mock calendar events
- day4/data/email_log.jsonl — mock email notifications

Day 5 — LangGraph Orchestration & Tool Calling

Deliverables

- day5/src/state.py — AgentState with 15 fields
- day5/src/graph.py — 9-node StateGraph with conditional routing
- day5/src/nodes.py — greeting, intent_detection, recommendation, rag, booking, rescheduling, cancellation, clarification, goodbye
- day5/src/tools.py — 8 tool wrappers bridging Day 2 RAG + Day 4 calendar/email/CRM
- day5/src/state_logger.py — @logged_node decorator
- day5/src/run_demo.py — 5 scripted conversations

Validation Rules Enforced (Task 4)

- ✓ Never book unavailable slots (availability checked before booking)
- ✓ Never recommend unavailable properties (only shows SQL results)
- ✓ Ask clarification instead of guessing (see “vague_then_clarify”)

Test Results

Conversation	Outcome
buyer_full_flow	3 properties recommended, booking successful
rental_reschedule_cancel	All 4 intents routed correctly
vague_then_clarify	Clarification requested when city missing
impossible_budget	Honest “no properties”, no hallucination
faq_then_goodbye	RAG answer + polite goodbye

Execution Traces

day5/data/execution_traces.json — every node transition logged.

Day 6 — Testing, Evaluation & Security

Task 1: Evaluation Suite (48 Conversations)

- Success rate: 100% (48/48)
- Categories: buyer, buyer_objection, renter, investor, commercial, reschedule, cancel, faq, off_topic, prompt_injection, angry, silent_caller, returning, multi_intent, goodbye

Task 2: Prompt Injection Testing (12 Attacks)

- Blocked: 12/12 (100%)
- Attack types: instruction override, prompt reveal, role hijack, fake booking, API key extraction, database dump, PII extraction, jailbreak (DAN), unicode obfuscation, fake tool calls, internal pricing probe

Task 3: Performance Evaluation

- Latency: avg=47.9ms, p50=4.9ms, p95=412.2ms → PASS
- RAG accuracy: 5/5 (100%)

Task 4: Monitoring Plan

- Metrics: latency, voice quality, API failure rates, booking success, RAG miss rate, injection attempts
- Alerts: P95 > 3500ms → PagerDuty; any successful injection → PagerDuty

Task 5: Deployment Readiness

- Docker, FastAPI, environment variables, logging, health checks

Artifacts

- day6/data/test_results.json — 48/48 conversations
- day6/data/security_results.json — 12/12 attacks blocked
- day6/data/performance_results.json — latency + RAG data
- day6/docs/monitoring_plan.md — full monitoring plan

Day 7 — Capstone: Deployment, Presentation & Handover

Task 1: Production Deployment

FastAPI backend (day7/app/main.py) — endpoints:

- GET /health — liveness check
- POST /chat — conversational turn
- GET /sessions/{session_id} — session state
- GET /docs — interactive Swagger UI
- Docker + docker-compose — one-command deploy
- Health check endpoint at /health
- Deployable to: Railway, Render, AWS, Azure

Task 2: Executive Documentation

- day7/docs/README.md — Quickstart, architecture, results
- day7/docs/API.md — Complete API reference
- day7/docs/USER_GUIDE.md — Sample conversations, tips
- day7/docs/MAINTENANCE.md — Weekly/monthly tasks, backups
- day7/docs/TROUBLESHOOTING.md — Common errors + fixes

Task 3: Monitoring & Maintenance Plan

- Latency thresholds: P95 < 2000ms
- Uptime target: 99.5%
- Weekly vector DB refresh
- Monthly backup + key rotation
- Security review cadence

Task 4: Stakeholder Demonstration

- day7/docs/demo_script.md — 10-minute walkthrough with 4 live demos
- day7/demo/slides_outline.md — 12-slide deck
- 5 screenshots (buyer query, booking, security, tests, Swagger UI)

Task 5: Future Enhancements

- WhatsApp + SMS integration
- Live MLS/property feed
- Punjabi + English multilingual
- Voice cloning for brand representatives
- Analytics dashboard + lead scoring
- Payment gateway integration
- CRM integrations (Salesforce, HubSpot)
- Automatic follow-up campaigns

API Test Results

tests/test_api.py::test_health PASSED

[25%]

```
tests/test_api.py::test_chat_buyer PASSED [ 50%]  
tests/test_api.py::test_chat_multi_turn PASSED [ 75%]  
tests/test_api.py::test_injection_blocked PASSED [100%]  
===== 4 passed in 12.52s =====
```

Consolidated Evaluation Results

Metric	Result	Target	Status
Retrieval accuracy	90%	>85%	✓
Chunk precision@1	80%	>75%	✓
Test conversations	48/48 (100%)	>90%	✓
Prompt injections blocked	12/12 (100%)	100%	✓
Latency p95	412 ms	<2000 ms	✓
RAG accuracy	100%	>95%	✓
API tests	4/4 (100%)	100%	✓
Human naturalness	4.0/5	>3.5	✓
Human persuasiveness	4.0/5	>3.5	✓
Human fluency	4.0/5	>3.5	✓

Architecture

Caller → STT (Deepgram) → LLM (Groq) → RAG (SQL + Vector)
→ LangGraph Agent (9 nodes) → Calendar / Email / CRM
→ TTS (Fish Audio) → Caller

Tech Stack

- LLM: Groq (GPT-OSS-120B)
- STT: Deepgram Nova-3
- TTS: Fish Audio
- Agent: LangGraph
- Vector DB: ChromaDB
- Structured DB: SQLite
- Backend: FastAPI
- Automation: n8n + Python workflow
- Deployment: Docker

Limitations

- Calendar/Email default to mock mode — live mode requires OAuth setup
- Voice pipeline simulates STT/TTS audio I/O — real audio hardware needed for live phone calls
- Cold-start latency 3-5s — production deployments load models at startup
- Sample size 700 properties — production would use full MLS feed
- Duplicate greeting in trace — cosmetic (no functional impact)

Future Enhancements

Next 3 Months

- WhatsApp + SMS integration
- Punjabi + English multilingual support

Next 6 Months

- Live MLS feed
- Voice cloning for brand representatives
- Analytics dashboard + lead scoring

Next 12 Months

- Payment gateway integration
- Multi-tenant SaaS offering
- Predictive property valuation AI

Submission Checklist

- ✓ Working AI Voice Agent with fluent UrduLish
- ✓ Complete LangGraph workflow + tool orchestration
- ✓ RAG pipeline with vector DB
- ✓ Structured property DB + recommendation logic
- ✓ Google Calendar integration (mock mode ready for live)
- ✓ Email automation
- ✓ CRM-ready logging
- ✓ FastAPI backend with documented endpoints
- ✓ Comprehensive evaluation report
- ✓ Prompt injection & security testing
- ✓ Monitoring & maintenance plan
- ✓ Dockerized deployment
- ✓ Complete documentation (README, API, User, Maintenance, Troubleshooting)
- ✓ 10-minute demo script + slide deck
- ✓ 5 screenshots (buyer, booking, security, tests, Swagger)